BF gates in Tensorflow Quantum

https://github.com/tensorflow/quantum/issues/709

ROW 2

BF gates in Tensorflow Quantum #709

Closed

RubensZimbres opened on Aug 13, 2022 · edited by RubensZimbres

Edits

Hello, I want to develop a neural network considering noise from a real quantum machine with cirq, considering the following Circuit:

`

```
def make_GHZ_withbitflips(num_qubits,measurements=True):
myqubits = cirq.LineQubit.range(num_qubits)
GHZ_circuit = cirq.Circuit([
cirq.Moment([cirq.H(myqubits[0])]),
cirq.rx(a).on(q0),
```

```
cirq.ry(b).on(q1),cirq.ry(a).on(q2), cirq.CNOT(control=q2, target=q3), cirq.CNOT(control=q1, target=q0)
])

for x in range(num_qubits-1):
    GHZ_circuit.append([cirq.CNOT(myqubits[x],myqubits[x+1]),cirq.bit_flip(p=0.1)(myqubits[x+1])])

if measurements:
    GHZ_circuit.append(cirq.Moment(cirq.measure_each(*myqubits)))
return GHZ_circuit
```

`

However, it seems that Tensorfow Quantum does not support BF gates, is this right ?

I'm using Tensorflow Quantum 0.6.1 and cirq==1.0.0

The code that is not supported is: `cirq.bit_flip(p=0.1)(myqubits[x+1])`

The error:

```
{ "name": "InvalidArgumentError", "message": "Exception encountered when calling layer \"expectation_4\" (type
Expectation).\n\nCould not parse gate id: BF. This is likely because a cirq.Channel was used in an op that
does not support them. [Op:TfqSimulateExpectation]\n\nCall arguments received:\n  • inputs=tf.Tensor(shape=
(2,), dtype=string)\n  • symbol_names=tf.Tensor(shape=(2,), dtype=string)\n  • symbol_values=tf.Tensor(shape=
(2, 2), dtype=float32)\n  • operators=tf.Tensor(shape=(2, 2), dtype=string)\n  • repetitions=None\n  •
initializer=<keras.initializers.initializers_v2.RandomUniform object at 0x7f403e14c310>", "stack":
"\u001b[0;31m---------------------------------------------------------------------------
\u001b[0m\n\u001b[0;31mInvalidArgumentError\u001b[0m                      Traceback (most recent call
last)\n\u001b[1;32m/home/theone/other_models/Quantum_Start/Quantum_ML_try_FOI_work3_noise.ipynb Cell
48\u001b[0m in \u001b[0;36m<cell line: 1>\u001b[34m()\u001b[0m\n\u001b[0;32m----> <a href='vscode-notebook-
cell:/home/theone/other_models/Quantum_Start/Quantum_ML_try_FOI_work3_noise.ipynb#X64sZmlsZQ%3D%3D?
line=0'>1</a>\u001b[0m model([datapoint_circuits, commands])\u001b[39m.\u001b[39mnumpy()\n\nFile
\u001b[0;32m~/.conda/envs/tf27/lib/python3.9/site-packages/keras/utils/traceback_utils.py:67\u001b[0m, in
\u001b[0;36mfilter_traceback.<locals>.error_handler\u001b[0;34m(*args, **kwargs)\u001b[0m\n\u001b[1;32m
65\u001b[0m \u001b[39mexcept\u001b[39;00m \u001b[39mException\u001b[39;00m \u001b[39mas\u001b[39;00m e:
\u001b[39m# pylint: disable=broad-except\u001b[39;00m\n\u001b[1;32m     66\u001b[0m   filtered_tb
\u001b[39m=\u001b[39m _process_traceback_frames(e\u001b[39m.\u001b[39m__traceback__)\n\u001b[0;32m--->
67\u001b[0m   \u001b[39mraise\u001b[39;00m e\u001b[39m.\u001b[39mwith_traceback(filtered_tb)
\u001b[39mfrom\u001b[39;00m \u001b[39mNone\u001b[39m\n\n\u001b[1;32m     68\u001b[0m
\u001b[39mfinally\u001b[39;00m:\n\u001b[1;32m     69\u001b[0m   \u001b[39mdel\u001b[39;00m filtered_tb\n\nFile
\u001b[0;32m~/.conda/envs/tf27/lib/python3.9/site-
packages/tensorflow_quantum/python/layers/high_level/controlled_pqc.py:264\u001b[0m, in
\u001b[0;36mControlledPQC.call\u001b[0;34m(self, inputs)\u001b[0m\n\n\u001b[1;32m    261\u001b[0m \u001b[39m#
this is disabled to make autograph compilation easier.\u001b[39;00m\n\u001b[1;32m    262\u001b[0m \u001b[39m#
pylint: disable=no-else-return\u001b[39;00m\n\n\u001b[1;32m    263\u001b[0m \u001b[39mif\u001b[39;00m
\u001b[39mself\u001b[39m\u001b[39m.\u001b[39m_analytic:\n\u001b[0;32m--> 264\u001b[0m
\u001b[39mreturn\u001b[39;00m
\u001b[39mself\u001b[39;49m\u001b[39m.\u001b[39;49m_layer(model_appended,\n\u001b[1;32m    265\u001b[0m
symbol_names\u001b[39m=\u001b[39;49m\u001b[39mself\u001b[39;49m\u001b[39m.\u001b[39;49m_symbols,\n\u001b[1;32m
   266\u001b[0m
symbol_values\u001b[39m=\u001b[39;49minputs[\u001b[39m1\u001b[39;49m],\n\u001b[1;32m    267\u001b[0m
            operators\u001b[39m=\u001b[39;49mtiled_up_operators)\n\u001b[1;32m    268\u001b[0m
```

```
270\u001b[0m                                    [circuit_batch_dim, \u001b[39m1\u001b[39m])\n\nFile
\u001b[0;32m~/.conda/envs/tf27/lib/python3.9/site-
packages/tensorflow_quantum/python/layers/circuit_executors/expectation.py:356\u001b[0m, in
\u001b[0;36mExpectation.call\u001b[0;34m(self, inputs, symbol_names, symbol_values, operators, repetitions,
initializer)\u001b[0m\n\u001b[1;32m    353\u001b[0m     \u001b[39mreturn\u001b[39;00m
\u001b[39mself\u001b[39m\u001b[39m.\u001b[39m_expectation_op(inputs, symbol_names,
symbol_values,\n\u001b[1;32m    354\u001b[0m                                    operators,
num_samples)\n\u001b[1;32m    355\u001b[0m \u001b[39melse\u001b[39;00m:\n\u001b[0;32m--> 356\u001b[0m
\u001b[39mreturn\u001b[39;00m \u001b[39mself\u001b[39;49m\u001b[39m.\u001b[39;49m_expectation_op(inputs,
symbol_names, symbol_values,\n\u001b[1;32m    357\u001b[0m                                    operators)\n\nFile
\u001b[0;32m~/.conda/envs/tf27/lib/python3.9/site-
packages/tensorflow_quantum/python/differentiators/differentiator.py:156\u001b[0m, in
\u001b[0;36mDifferentiator.generate_differentiable_op.<locals>.op_wrapper_analytic\u001b[0;34m(programs,
symbol_names, symbol_values, pauli_sums)\u001b[0m\n\u001b[1;32m    153\u001b[0m
\u001b[39m@tf\u001b[39m\u001b[39m.\u001b[39mcustom_gradient\n\u001b[1;32m    154\u001b[0m
\u001b[39mdef\u001b[39;00m \u001b[39mop_wrapper_analytic\u001b[39m(programs, symbol_names,
symbol_values,\n\u001b[1;32m    155\u001b[0m                                    pauli_sums):\n\u001b[0;32m-->
156\u001b[0m     forward_pass_vals \u001b[39m=\u001b[39m analytic_op(programs, symbol_names,\n\u001b[1;32m
157\u001b[0m                                    symbol_values, pauli_sums)\n\u001b[1;32m    159\u001b[0m
\u001b[39mdef\u001b[39;00m \u001b[39mgradient\u001b[39m(grad):\n\u001b[1;32m    160\u001b[0m
\u001b[39mreturn\u001b[39;00m \u001b[39mself\u001b[39m\u001b[39m.\u001b[39m_differentiate_ana(programs,
symbol_names,\n\u001b[1;32m    161\u001b[0m                                    symbol_values,
pauli_sums,\n\u001b[1;32m    162\u001b[0m                                    forward_pass_vals,
grad)\n\nFile \u001b[0;32m~/.conda/envs/tf27/lib/python3.9/site-
packages/tensorflow_quantum/core/ops/circuit_execution_ops.py:135\u001b[0m, in \u001b[0;36mget_expectation_op.
<locals>.<lambda>\u001b[0;34m(programs, symbol_names, symbol_values, pauli_sums)\u001b[0m\n\u001b[1;32m
131\u001b[0m \u001b[39mif\u001b[39;00m op \u001b[39mis\u001b[39;00m \u001b[39mnot\u001b[39;00m
\u001b[39mNone\u001b[39;00m:\n\u001b[1;32m    132\u001b[0m     \u001b[39mif\u001b[39;00m quantum_concurrent
\u001b[39mis\u001b[39;00m \u001b[39mTrue\u001b[39;00m:\n\u001b[1;32m    133\u001b[0m         \u001b[39m#
Return an op that does not block graph level parallelism.\u001b[39;00m\n\u001b[1;32m    134\u001b[0m
\u001b[39mreturn\u001b[39;00m \u001b[39mlambda\u001b[39;00m programs, symbol_names, symbol_values, pauli_sums:
\\\n\u001b[0;32m--> 135\u001b[0m             op(programs, symbol_names, symbol_values,
pauli_sums)\n\u001b[1;32m    137\u001b[0m     \u001b[39m# Return an op that does block graph level
parallelism.\u001b[39;00m\n\u001b[1;32m    138\u001b[0m     \u001b[39mreturn\u001b[39;00m
\u001b[39mlambda\u001b[39;00m programs, symbol_names, symbol_values, pauli_sums: \\\n\u001b[1;32m
139\u001b[0m
quantum_context\u001b[39m.\u001b[39m_GLOBAL_OP_LOCK\u001b[39m.\u001b[39mexecute(\u001b[39mlambda\u001b[39;00m:
op(\n\u001b[1;32m    140\u001b[0m             programs, symbol_names, symbol_values, pauli_sums))\n\nFile
\u001b[0;32m~/.conda/envs/tf27/lib/python3.9/site-
packages/tensorflow_quantum/core/ops/tfq_simulate_ops.py:44\u001b[0m, in
\u001b[0;36mtfq_simulate_expectation\u001b[0;34m(programs, symbol_names, symbol_values,
pauli_sums)\u001b[0m\n\u001b[1;32m     22\u001b[0m \u001b[39mdef\u001b[39;00m
\u001b[39mtfq_simulate_expectation\u001b[39m(programs, symbol_names, symbol_values, pauli_sums):\n\u001b[1;32m
    23\u001b[0m     \u001b[39m\"\"\"Calculate the expectation value of circuits wrt some
operator(s)\u001b[39;00m\n\n\u001b[1;32m     24\u001b[0m \n\u001b[1;32m     25\u001b[0m \u001b[39m
Args:\u001b[39;00m\n\n\u001b[0;32m   (...)\u001b[0m\n\n\u001b[1;32m     42\u001b[0m \u001b[39m           (after
resolving the corresponding parameters in).\u001b[39;00m\n\n\u001b[1;32m     43\u001b[0m \u001b[39m
\"\"\"\u001b[39;00m\n\n\u001b[0;32m---> 44\u001b[0m     \u001b[39mreturn\u001b[39;00m
SIM_OP_MODULE\u001b[39m.\u001b[39;49mtfq_simulate_expectation(\n\u001b[1;32m     45\u001b[0m         programs,
symbol_names, tf\u001b[39m.\u001b[39;49mcast(symbol_values, tf\u001b[39m.\u001b[39;49mfloat32),
pauli_sums)\n\nFile \u001b[0;32m<string>:48\u001b[0m, in
\u001b[0;36mtfq_simulate_expectation\u001b[0;34m(programs, symbol_names, symbol_values, pauli_sums,
name)\u001b[0m\n\n\n\u001b[0;31mInvalidArgumentError\u001b[0m: Exception encountered when calling layer
\"expectation_4\" (type Expectation).\n\nCould not parse gate id: BF. This is likely because a cirq.Channel
was used in an op that does not support them. [Op:TfqSimulateExpectation]\n\nCall arguments received:\n  •
inputs=tf.Tensor(shape=(2,), dtype=string)\n  • symbol_names=tf.Tensor(shape=(2,), dtype=string)\n  •
symbol_values=tf.Tensor(shape=(2, 2), dtype=float32)\n  • operators=tf.Tensor(shape=(2, 2), dtype=string)\n  •
repetitions=None\n  • initializer=<keras.initializers.initializers_v2.RandomUniform object at 0x7f403e14c310>"
}
```

which will return (ignore the values):

```
{cirq.amplitude_damp(gamma=0.01): 1,
 cirq.asymmetric_depolarize(error_probabilities={'I': 0.94, 'X': 0.01, 'Y': 0.02, 'Z': 0.03}): 1,
 cirq.bit_flip(p=0.01): 1,
 cirq.depolarize(p=0.01): 1,
 cirq.generalized_amplitude_damp(p=0.01,gamma=0.02): 1,
 cirq.phase_damp(gamma=0.01): 1,
 cirq.phase_flip(p=0.01): 1,
 cirq.ResetChannel(): 1}
```

I just verified this with something similar to your GHZ example:

```
import tensorflow_quantum as tfq
import cirq

def make_GHZ(qubits):
  c = cirq.Circuit()
  c += cirq.H(qubits[0])
  for i in range(len(qubits) - 1):
    c += cirq.CNOT(qubits[i], qubits[i + 1])

  return c

qs = [cirq.GridQubit(0, i) for i in range(3)]
noise_test = make_GHZ(qs).with_noise(cirq.bit_flip(p=0.05))
noiseless_test = make_GHZ(qs)
ops = [cirq.Z(i) for i in qs]

exp = tfq.layers.Expectation()(noiseless_test, symbol_names=[], symbol_values=[[]], operators=ops)
print(exp)
exp = tfq.layers.SampledExpectation(backend='noisy')(noise_test, symbol_names=[], symbol_values=[[]], operato
print(exp)
```

Which returned

```
tf.Tensor([[0. 0. 0.]], shape=(1, 3), dtype=float32)
tf.Tensor([[ 0.002 -0.024  0.028]], shape=(1, 3), dtype=float32)
```

For more information on simulating noise with TFQ, I recommend: https://www.tensorflow.org/quantum/tutorials/noise

I would guess there is something else in the code causing your error, but I cannot tell from what is provided. I would recommend linking to the minimal viable example to demonstrate the error. Additionally, you can format the code neater using three of the ` symbols e.g. ```python CODE ```

RubensZimbres on Aug 15, 2022 · edited by RubensZimbres Edits Author ···

Thanks very much, I will work on that.

UPDATE: I used `tfq.layers.NoisyControlledPQC` and it worked.